TEXORA · SESSIONAL NOTES · LAB EXPERIMENT SHEET

Problem Formulation, Problem Solving and Flowcharts

Before any code: stating a problem exactly — its inputs, outputs and rules — then solving it on paper with an algorithm and a flowchart whose symbols mean one thing each, so the program that follows is a translation, not an invention.

CSE-2106 · FUNDAMENTALS OF COMPUTING SESSIONAL · EXPERIMENT 3 · LEVEL 2 TERM 1

1. Aim

To formulate simple problems precisely, express their solutions as step-by-step algorithms (pseudocode), and draw correct flowcharts using the standard symbols.

2. Concepts you need

Problem formulation answers four questions before solving begins: What is **given** (inputs and their types/ranges)? What must be **produced** (outputs)? What **rules** connect them (formulas, conditions)? What are the **limits** (special cases — zero, negatives, ties)? Most "programming errors" are formulation errors discovered late.

An **algorithm** is a finite sequence of unambiguous steps that reaches the output for every valid input. A **flowchart** is the algorithm drawn; each symbol has exactly one job:

SYMBOL	NAME	MEANING
Rounded rectangle (stadium)	Terminator	Start / End — exactly one entry at the top, flow leaves downward
Parallelogram	Input / Output	Read data in, or print results out
Rectangle	Process	A calculation or assignment: Sum = Sum + i
Diamond	Decision	A yes/no question; two exits, each labelled (Yes/No or True/False)
Arrow (flow line)	Flow	Direction of execution; a loop is an arrow that returns to an earlier symbol
Circle	Connector	Joins a chart split across pages or columns

Rules of the craft: one flow line into a symbol (except the merge after a decision), every decision exit labelled, and the chart must be **traceable** — for any test data you can walk it with a pencil and arrive at the answer.

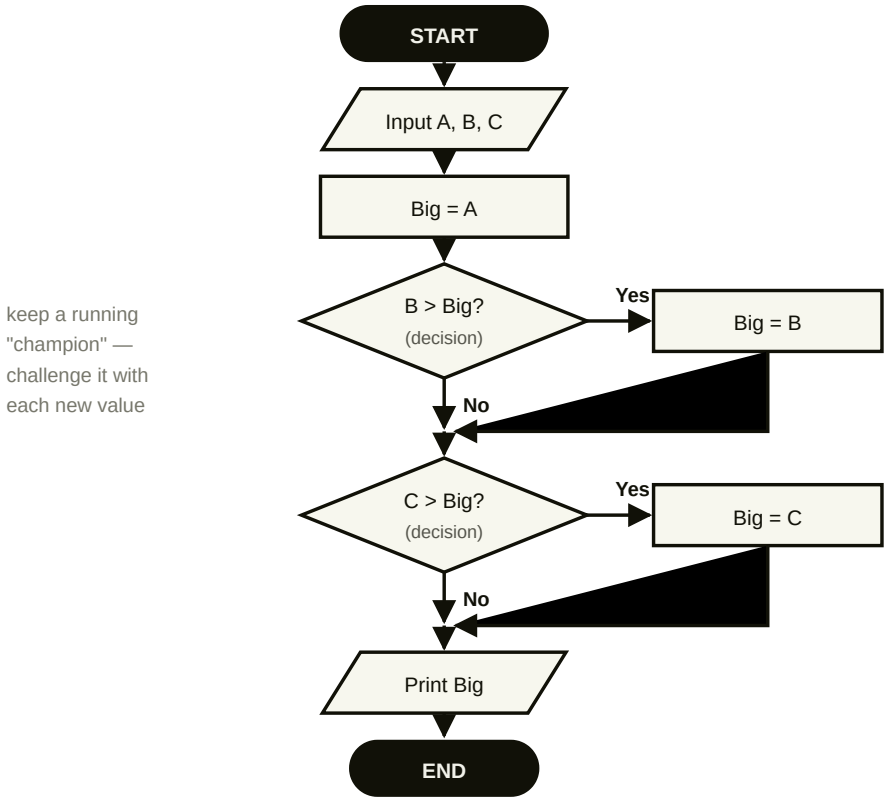


Figure 1 — Flowchart: largest of three numbers. Note the discipline: every diamond has exactly two labelled exits, and both branches merge before the next symbol.

3. Lab procedure

1. For each problem given, first write the formulation: inputs, outputs, rules, limits. Do not draw yet.
2. Write the algorithm as numbered pseudocode steps.
3. Draw the flowchart on paper using the standard symbols; check the rule list (one entry, labelled exits, terminators at both ends).
4. **Trace** the chart with the test data supplied, writing the value of every variable after each step in a trace table. If the trace fails, the chart — not the data — is wrong.
5. Redraw the corrected chart neatly (in a drawing tool or office shapes) and keep the trace table with it as evidence.

4. Worked examples

Example A — Largest of three numbers (Figure 1)

Formulation: inputs — three numbers A, B, C. Output — the largest. Rule — keep a running maximum ("champion") and challenge it with each remaining value. Limit — ties: if two are equal and largest, reporting that value is correct.

Trace with A = 12, B = 25, C = 9: Big = 12 → B > Big (25 > 12) Yes → Big = 25 → C > Big (9 > 25) No → Print **25**. ✓

Example B — Sum of the first N natural numbers (a loop)

Formulation: input — N. Output — 1 + 2 + … + N. Rule — accumulate: start Sum = 0 and i = 1; while i ≤ N, add i to Sum and increase i by 1. The flowchart is the champion chart's skeleton with its decision exit wired *backwards*: the "i ≤ N? Yes" arrow returns to the process box, and "No" falls through to Print Sum.

Trace with N = 5: (i, Sum) runs (1,0) → add → (2,1) → (3,3) → (4,6) → (5,10) → add → (6,15); i ≤ N fails at i = 6 → Print **15**. ✓ (Check by formula: N(N+1)/2 = 5×6/2 = 15 — an algorithm and a formula agreeing is the strongest verification a beginner has.)

Example C — Even or odd

Input a number n; decision "n mod 2 = 0?" — Yes → Print "Even"; No → Print "Odd". One diamond, two output parallelograms, one merge, End. Trace: n = 17 → 17 mod 2 = 1 ≠ 0 → **Odd**.

5. Practice tasks

1. Formulate and draw the flowchart for computing the factorial of a number N (N! = 1 × 2 × … × N). Include the formulation (inputs, output, rule, and the limit case N = 0) and a trace for N = 4.
2. Draw the flowchart that reads a student's marks and prints "Pass" if marks ≥ 40, otherwise "Fail". Trace it for marks = 35 and marks = 40.
3. A flowchart for Example B is drawn with the test "i ≤ N?" placed *after* the "Sum = Sum + i" box instead of before it. Trace both versions for N = 0 and state the difference in behaviour and its name.

Answer key

1. **Formulation:** input N (whole number ≥ 0); output N!; rule — Fact = 1; for i from 1 to N, Fact = Fact × i; limit — 0! is defined as 1, and the loop must give that (with Fact starting at 1 and the loop never running, it does). **Flowchart:** Start → Input N → Fact = 1, i = 1 → decision "i ≤ N?" — Yes → Fact = Fact × i, i = i + 1 → loop back to the decision; No → Print Fact → End. **Trace N = 4:** Fact runs 1 → (i=1) 1 → (i=2) 2 → (i=3) 6 → (i=4) 24; i = 5 fails the test → Print **24**.

2. Start → Input marks → decision "marks ≥ 40?" — Yes → Print "Pass"; No → Print "Fail" → merge → End. Traces: 35 → 35 ≥ 40 false → **Fail**; 40 → 40 ≥ 40 true → **Pass** (the boundary belongs to Pass because the condition uses ≥).
3. With the test after the process box, the body runs once before any test: for N = 0 it still executes Sum = Sum + 1 and ends by printing 1 — a wrong answer produced without any error message. The test-first version prints 0 correctly. A loop whose test comes first is a **pre-test** loop (it may run zero times); test-last is **post-test** (always runs at least once). Choosing between them is a formulation decision, not a drawing preference.
